

what is node js ----->>>

node js is not a language. Node js is async language

this is a Server run time Environment (run JavaScript on server environment)

Node js can connect with database

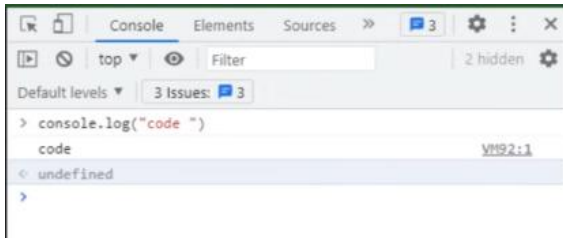
Code and Syntax is very similar to JavaScript, But not exact same

Node js free, open source

Node js use Chrome's V8 engine to execute code

Node js Version check ----->>> `node -v`

Npm js Version check ----->>> `npm -v`



With this example result shows undefined because with console.log only print the message not return anythings

```
> var a = 50
< undefined
> var b = 40
< undefined
> c = a+b
< 90
>
```

Here at the last equation result shows without undefined **because c is return the plus value of a and b**

Run node js from folder: ----->>>

Create a folder.

In side folder create index.js file.



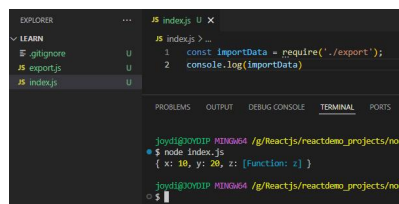
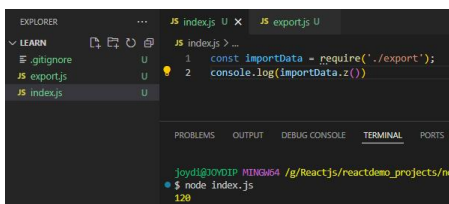
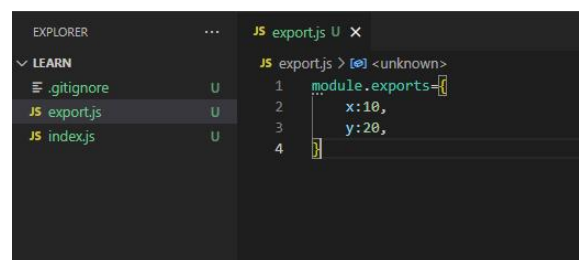
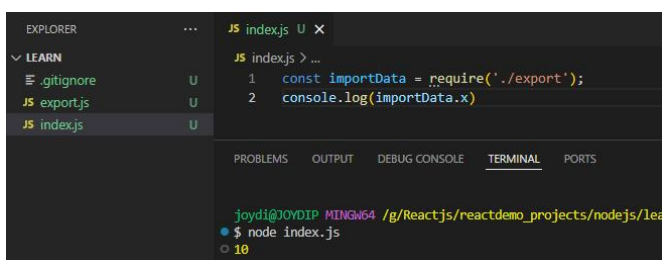
Difference between browser console and node js console ----->>>

Both console are different

browser console is a function of browser

node js console is a internal module of node js because node is not run in browser

Import file in node js



What is filter: -----

Filter is a function which helps to find a specific data from a array. It always works with array data only.

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 arr.filter((item)=>{
3   console.log(item)
4 })
```

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects
$ node filter.js
2
3
4
5
6
8
9
23
```

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item===5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 5 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

```
JS filter.js U X
JS filter.js > filterResult > arr.filter() callback
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item > 5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 6, 8, 9, 23 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

Types of module in Node JS -----

1. Core Module > 1.1> Global Module, 1.2> Non Global Module
2. External Module

In Node.js, **core modules** are built-in modules that come pre-packaged with the Node.js runtime. They provide essential functionality for various operations, such as working with files, streams, HTTP servers, cryptography, and more. These modules are directly available without requiring any additional installation or setup.

Characteristics of Core Modules

- ◆ **Pre-installed:** They are bundled with Node.js and don't require installation through npm.
- ◆ **Lightweight:** Optimized for performance and directly integrated with Node.js.
- ◆ **Always Available:** You can access them by simply using the `require()` function.

List of Node.js Core Modules: Other notable core modules include:

- ◆ **Buffer:** For working with binary data.
- ◆ **Stream:** For handling streaming data.
- ◆ **URL:** For URL resolution and parsing.
- ◆ **Timers:** For managing timeouts and intervals.
- ◆ **Zlib:** For compression and decompression.

Why Use Core Modules?

- ◆ No need to install additional dependencies.
- ◆ Highly efficient and optimized for Node.js.
- ◆ Reduce external dependencies in your project.

Global modules -----

In Node.js, global modules refer to Globally Installed Modules. These are modules installed system-wide and can be accessed from any project on your machine. They are not tied to a specific project directory.

To install a module globally, use the `-g` flag with npm:

Example: `npm install -g <module_name>`

Make a basic server and output on server: -----

In Node.js, the **http module** is a **core module** used to create HTTP servers and clients. It provides essential functionalities to interact with the **HTTP protocol**, enabling Node.js applications to handle web requests and responses.

Key Features of the http Module

- ◆ **Core Module:** Pre-installed with Node.js, no need to install it separately.
- ◆ **Web Server Creation:** You can create an HTTP server to handle client requests.
- ◆ **HTTP Client:** Allows making HTTP requests to external servers.
- ◆ **Customizable:** Offers flexibility to define how the server handles incoming requests and sends responses.

Basic Usage of the http Module

```
JS basic_server.js U X
JS basic_server.js > server.listen('4300') callback
1  const http = require('http');
2
3  const server = http.createServer((req, resp) => {
4    resp.write("This is a simple created server");
5    resp.end();
6  });
7
8  server.listen('4300', () => {
9    console.log("server is running on port: 4300");
10 });
11
12
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/
$ node basic_server.js
server is running on port: 4300
```

```
← → ↺ ⓘ localhost:4300
⌵ | 🌐 Discord | Friends | 📧 Gmail | 📁 All Tools
This is a simple created server
```

what is package.json -----

In Node.js, **package.json** is a file that provides metadata about a project or module. It serves as the central configuration file for a Node.js project and includes information about the project, its dependencies, scripts, and other settings.

Creating a package.json File

create a package.json file using the following command: **npm init**

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

Customize Package.json

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["nodejs", "example", "app"],
  "author": "author name",
  "license": "ISC"
}
```

```
"scripts": {
  "start": "node index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

With this above start key we can run code by command: **npm start**

Create a Simple API: -----

```
const http = require("http");
http.createServer((req, resp)=>{
  resp.writeHead(200, { 'content-type' : 'application/json' });
  resp.write(JSON.stringify({ "name": "nodejs", "type": "javascript" }));
  resp.end();
}).listen(5000);
```

Pretty-print ☐

```
{"name":"nodejs","type":"javascript"}
```

Get Simple API Data from other file : -----

```
JS simple_api_datajs U X
JS simple_api_datajs > ...
1  const data = [
2    { "name": "nodejs", "type": "javascript" },
3    { "name": "reactjs", "type": "javascript" },
4    { "name": "angular", "type": "javascript" },
5  ];
6
7  module.exports = data;
```

```
JS simple_apijs U X
JS simple_apijs > http.createServer() callback
1  const http = require("http");
2  http.createServer((req, resp)=>{
3    resp.writeHead(200, { 'content-type' : 'application/json' });
4    resp.write(JSON.stringify(data));
5    resp.end();
6  }).listen(5000);
7
```

Pretty-print ☐

```
[{"name":"nodejs","type":"javascript"}, {"name":"reactjs","type":"javascript"}, {"name":"angular","type":"javascript"}]
```

Get input from command line : -----

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js hello hi
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js',
  'hello',
  'hi'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv[2]);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/
$ node command_line_get_input.js hello hi
hello
```

